

Topic 1: Statistics & Probability for Detection

By hand first, then in Python: describe normal traffic, measure surprise, and count what an alert is worth

Duration	about 2 hours
Module	CITY2161 Data Modelling and Machine Learning for Cyber Security (FHEQ Level 5)
Tools	Python 3.12+, pandas, NumPy, SciPy, matplotlib; cyberlab.py from the Lab Kit

This workshop follows the lecture's three questions on the module dataset. Each part starts on paper, because a number you have calculated yourself is a number you can explain, and then checks the answer in Python. Keep your hand working and your printouts for your portfolio.

SETUP

Follow the Lab Kit (00_Lab_Kit.pdf) once: create the virtual environment and save cyberlab.py next to your notebook. Every workshop starts with the same two lines — `import cyberlab as C`, then `X = C.load()` — which give you the 7,200 host-hour table. `X.label` is 1 for attack hours and 0 for normal ones.

Part A — What is normal? (30 min)

1. On paper: for the nine hours 21, 5, 47, 25, 14, 13, 14, 8, 22, work out the mean, the median, the range and the interquartile range, using the method in Worked example 1. Check your answers against the notes.
2. In Python: select the normal hours (`X.label == 0`) and compute the median, the quartiles, and the 95th and 99th percentiles of `n_flows`. Plot a histogram.
3. Classify the columns `n_flows`, `top_service`, `total_bytes` and `failed_logins` as count/measurement or category, and name one calculation that would be meaningless on the category.

```
1 import matplotlib.pyplot as plt
2 import cyberlab as C
3
4 X = C.load()
5 flows = X.loc[X.label == 0, "n_flows"]          # normal host-hours only
6 print(flows.median(), flows.quantile([0.25, 0.75, 0.95, 0.99]))
7 flows.hist(bins=range(0, 105, 5))
8 plt.xlabel("flows per normal host-hour")
```

Starter (Python): summarising normal traffic.

HINT

pandas works out quartiles by interpolating between values, so for the nine hours it gives 13.0 and 22.0, not your hand values. Both are fine; say which method you used.

PORTFOLIO CAPTURE

Keep your hand working, the percentile printout and the histogram, with one sentence on what a 'normal hour' looks like.

Part B — How surprising is it? (25 min)

1. For thresholds of 30, 45, 60 and 100 flows, count how many normal hours reach at least that many, and divide by the number of normal hours. Compare with Table 2 of the notes.
2. Find the host-hour with the most flows in the whole dataset. How many normal hours had at least that many? Was it an attack?

3. Check the Poisson model: compare the variance of normal `n_flows` with its mean. Then compute the Poisson probability of 60 or more flows with `scipy.stats.poisson.sf(59, mean)` and compare it with your count. Which would you trust for an alert rule, and why?

```

1 from scipy import stats
2
3 share = (flows >= 60).mean()           # True counts as 1, so the mean is the share
4 print("counted:", share)
5 print("mean", flows.mean(), "variance", flows.var())
6 print("Poisson says:", stats.poisson.sf(59, flows.mean())) # sf(59) is P(at least 60)

```

Starter (Python): a probability by counting, and by a model.

PORTFOLIO CAPTURE

Save your threshold table and a two-sentence verdict on the Poisson model.

Part C — Given an alert, is it an attack? (40 min)

1. On paper: a detector catches 95% of attacks and wrongly alerts on 2% of normal host-hours; attacks are 1 in 5,000. Draw a frequency tree for 100,000 host-hours and work out how many alerts are raised and what share are real.
2. In Python: write a function `real_share(base_rate, detection, false_alarm)` that returns the share of alerts that are real, using counts of 100,000 host-hours. Check it against your tree and against Table 3 of the notes.
3. Use your function to fill a table for base rates of 1 in 100, 1,000, 10,000 and 100,000. Then repeat with detection raised to 99%, and separately with the false-alarm rate halved. Which change helps more?
4. Simulate one day of 1,000,000 host-hours at a 1-in-10,000 base rate with NumPy random draws, count the alerts, and confirm the simulated share of real alerts is close to your function's answer.

```

1 def real_share(base_rate, detection, false_alarm, n=100_000):
2     attacks = n * base_rate
3     caught = attacks * detection
4     false_alerts = (n - attacks) * false_alarm
5     return caught / (caught + false_alerts)
6
7 for rate in (1/100, 1/1_000, 1/10_000, 1/100_000):
8     print(rate, round(real_share(rate, 0.90, 0.01), 4))

```

Starter (Python): counting what an alert is worth.

PORTFOLIO CAPTURE

Keep your hand-drawn tree, the base-rate table and your answer to 'which change helps more'. This is the key deliverable of the topic.

Part D — Checking many hosts at once (25 min + stretch)

1. Simulate 1,000 normal hosts, each with 30 hourly counts drawn from a Normal distribution with average 100 and standard deviation 15. For each host, test whether its average differs from 100 and count how many hosts are flagged at $\alpha = 0.05$. Is it close to 50?
2. Apply the Bonferroni correction ($\alpha = 0.05 \div 1,000$) and count again.
3. Add 10 flows per hour to 20 of the hosts. How many of those 20 does each threshold catch? Write two or three sentences on the trade-off for a team that can investigate twenty alerts a day.

```
1 import numpy as np
2 from scipy import stats
3
4 rng = np.random.default_rng(0)
5 counts = rng.normal(100, 15, size=(1000, 30))      # 1,000 hosts x 30 hours
6 z = (counts.mean(axis=1) - 100) / (15 / np.sqrt(30))
7 p = 2 * stats.norm.sf(np.abs(z))                  # one p-value per host
8 print("flagged at 0.05:", (p < 0.05).sum())
```

Starter (Python): 1,000 tests at once.

STRETCH GOAL

Add `scipy.stats.false_discovery_control(p)` — the Benjamini–Hochberg procedure — and flag hosts whose adjusted value is below 0.05. Compare alerts, attacks caught and the share of alerts that are real with Table 4 of the notes. Then, back in Part B, try a negative binomial model for the flow counts and see whether its tail matches the data better than the Poisson.

Submit your notebook, your hand working, the base-rate table and your Part D sentences as this topic's portfolio deliverable.